

KaamSetu Product Architecture and MVP Scope Lock v1.0

Product Architecture Overview

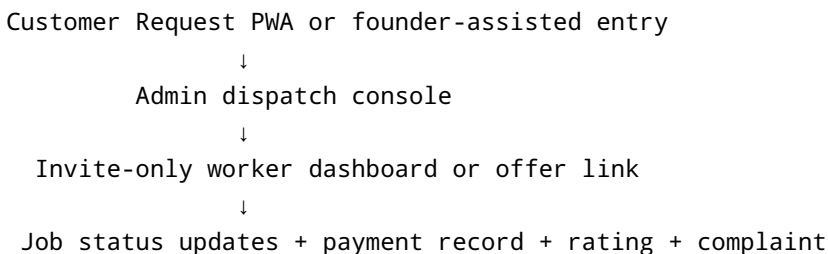
KaamSetu v1 should be built as a **dispatch-led hyperlocal operating system**, not as a full two-sided marketplace. In practical terms, that means customers should not browse hundreds of workers, compare public profiles, chat, negotiate endlessly, or watch moving maps. They should submit a request, get a tracking link, receive a worker assignment, see a small number of status changes, record payment, and rate the job. The recurring actors in the system are the **founder-admin** and the **verified worker network**. The customer side should stay extremely thin until the first 100–1000 jobs prove that repeat demand exists. That is the smallest shape that still satisfies the five core outcomes you specified: onboard workers, accept requests, match jobs, track jobs, record payments, and build trust.

The strongest architectural decision is to make KaamSetu v1 a **web-first PWA**, not a native Android/iOS app. A PWA can be installable, app-like, and single-codebase, and it avoids packaging, app-store review cycles, and store fees during the beta phase. That matters because Google Play requires a US\$25 one-time developer registration fee, while the Apple Developer Program is US\$99 per year, both of which are pure distraction before product-market proof. ¹

The second non-negotiable decision is to treat **WhatsApp, call support, and assisted booking as channels, not platform modules**. The free WhatsApp Business app is built for small businesses and is free to download, while the WhatsApp Business Platform is designed for larger-scale programmatic messaging and is charged on a per-message basis. The Business app is also fundamentally one-phone-number centric, which is exactly fine for a solo founder. For KaamSetu v1, the founder should use the free Business app manually and enter every assisted booking into the same admin workflow as app-originated bookings. ²

The third major decision is to **avoid native OTP, maps, and real-time chat** at launch. Firebase phone verification for production requires a billing account and phone authentication is billed per SMS/verification event, while Google Maps Platform uses pay-as-you-go pricing based on billable events. For a solo founder on a ₹10,000 early-build ceiling, both are unnecessary recurring cost centers. ³

A good mental model for the MVP is this:



↓
Single source of truth in one database

The architecture should therefore be:

Customer-facing PWA
Worker-facing PWA
Admin web panel
↓
Next.js web application
↓
Supabase Postgres + Storage
↓
Manual WhatsApp Business app communications
Optional Razorpay Payment Links only when prepayment is needed

The highest-level product thesis is simple: **KaamSetu v1 is not “an app for everyone”; it is “a job control system for one founder, one launch city, a small worker network, and the first 100 jobs.”**

MVP Scope Lock

Exact MVP scope. KaamSetu v1 should be an **invite-only, single-city, single-cluster, dispatch-led PWA**. It should support one founder-admin, a small verified worker network, assisted booking, customer request intake, job assignment, job status tracking, payment recording, ratings, and complaints. It should not try to solve catalogue discovery, dynamic marketplace pricing, automated WhatsApp, or mobile app distribution on day one.

Customer identity design. Customers should **not be forced into a full account system** in the MVP. The correct low-friction design is: phone number + invite code + request form + secure tracking link. That is enough for beta demand capture and removes the cost and complexity of consumer auth. Worker-side login matters. Customer-side login does not.

Category design. The schema should support categories generically, but the launch should be capped at **no more than five high-frequency categories** in one city cluster. The right categories are the ones with repeat local demand and short decision cycles, not the complete national catalogue. Over-broad category launch is how solo founders drown in empty supply and edge cases.

Pricing design. KaamSetu should support the three promised pricing modes in one unified job model:

- fixed_price
- quote_required
- daily_wage

Do **not** build three separate booking systems. Use one `pricing_mode` field and one `price_snapshot` structure per job. Fixed-price jobs use an admin-defined template. Quote jobs use one manual quote entry and one manual approval record. Daily-wage jobs use rate + duration + headcount, nothing more.

What to build now, what to reject, and what to delay.

Decision	Included items
Build now	Invite gating, worker onboarding, worker verification, customer request form, photo upload, assisted booking entry from admin, category/locality configuration, worker availability toggle, dispatch queue, worker accept/decline, job status updates, payment record, ratings, complaints, activity log
Reject now	Public worker discovery, open bids, live maps, real-time chat, in-app calling, wallet, escrow, coupons, referrals, ad system, IVR, dynamic pricing, worker social feed, native iOS app
Delay until 100 jobs	Customer login/history, push notifications, digital quote approval screen, self-serve rescheduling, second language pack beyond launch configuration, in-app payment links, simple B2B repeat booking templates
Delay until 1000 jobs	WhatsApp Cloud API, native Android app, live worker location, routing optimization, lead-fee logic, subscription billing, multi-city control panel, analytics warehouse, automated dispatch ranking, fleet-style operational dashboards

The “reject now” list is not cosmetic. It is what prevents the MVP from becoming a funded-startup fantasy. The clearest examples are native apps, WhatsApp API, maps, and OTP. Native apps add store fees and distribution overhead; WhatsApp Platform adds programmatic setup and message pricing; Maps are pay-as-you-go; OTP adds production billing and per-verification cost. All four should stay out until the first real demand loop is proven. ⁴

The most important scope lock is this sentence: **KaamSetu v1 is a workflow product, not a marketplace theatre product.** If a feature does not improve request capture, matching, execution, payment recording, or trust, it should not enter MVP.

User Surfaces and Job Workflows

Customer app screen list. In the MVP, the “customer app” is a customer-facing PWA, not a full authenticated consumer app.

Route or screen	Purpose
Landing and invite gate	Enter invite code or continue from assisted link
New request form	Category, locality, address text, timing, description
Photo upload	Up to 3 photos for context
Request submitted	Request ID, current status, support options

Route or screen	Purpose
Track job	Friendly status view, assigned worker details once confirmed
Completion and payment confirm	Mark amount paid and mode used
Rating and report issue	1–5 rating, complaint shortcut, notes

Worker app screen list. The worker app should be an invite-only PWA with a phone + PIN login.

Route or screen	Purpose
Login	Phone number + PIN
Profile and KYC	Name, photo, categories, localities, document upload
Dashboard	Availability status, pending offers, active jobs
Offer detail	Accept or decline offer, view job basics
Active job detail	Address, scope, timing, pricing mode, customer contact
Status update	On the way, started, completed, no-show, issue
Earnings and payment log	Closed jobs, amount collected, pending payment records

Admin panel screen list. The admin panel is the real MVP core.

Route or screen	Purpose
Admin login	Founder access only
Dashboard	New jobs, jobs in progress, complaints, daily summary
Request queue	Every incoming job, regardless of channel
Worker approvals	Review and approve worker onboarding
Worker directory	Filter by category, locality, status, trust level
Dispatch board	Candidate shortlist, offer history, assignment controls
Job detail view	Full ticket state, notes, payments, rating, complaint
Payment ledger	Cash, UPI, link payments, reconciliation notes
Settings	Categories, localities, invite codes, language copy, price templates

Booking flow. The booking flow should be deliberately boring, because boring flows are reliable.

A customer request should come into the system in only one of two ways: either the customer submits the PWA request form, or the founder creates the same request through the admin panel after a call or WhatsApp message. That means there is **one booking engine**, one job table, one state machine, and one

reporting path. There should never be a separate “WhatsApp booking system” or “assisted booking system” in code. Assisted booking is only a different `source_channel`, not a different workflow.

The exact flow should be: 1. Request created. 2. Job gets request ID and customer tracking token. 3. Admin reviews category, locality, urgency, and pricing mode. 4. Eligible workers are shortlisted. 5. Offer goes to up to three workers per round. 6. One worker is selected and assignment is confirmed. 7. Worker updates status. 8. Customer confirms completion and payment. 9. Customer rates or opens complaint.

Dispatch flow. Dispatch should be semi-manual in v1, not instant and algorithmic.

The right logic is: create a shortlist by `category + locality + availability + trust flag`, then allow the founder to dispatch to **no more than three workers in a round**. That avoids double-accept conflicts, reduces ghost assignments, and keeps quality control tight. If nobody accepts within the founder’s chosen response window, the request moves to the next round or is marked `no_worker_found`.

Worker-facing dispatch should be simple: the worker sees the offer, taps accept or decline, and nothing else. There should be no worker bidding, no chat thread, no quote countering, and no worker-visible competing offers.

Channel handling. WhatsApp and calls should stay manual in v1. The founder should use the free WhatsApp Business app for manual communication and use one business SIM for voice calls. Later, if inbound volume becomes too high for one person and multiple users need access, that is the trigger to evaluate the WhatsApp Business Platform. In the MVP, the free Business app is the correct fit. ⁵

Data Model and State Machines

Database tables required. The product schema should remain small. A good MVP schema is:

Table	Why it exists
<code>auth_users</code>	Framework-managed worker/admin auth records
<code>customer_profiles</code>	Name, phone, locality, basic customer identity
<code>worker_profiles</code>	Worker identity, service metadata, status, coverage
<code>worker_documents</code>	ID image, selfie, verification notes
<code>service_categories</code>	Category list and pricing-mode metadata
<code>localities</code>	Controlled locality names for the launch cluster
<code>jobs</code>	Core request and execution record
<code>job_media</code>	Job photos only, linked to job
<code>dispatch_attempts</code>	Worker offer history and response trail
<code>payments</code>	Cash, UPI, payment-link records

Table	Why it exists
<code>ratings</code>	Post-job customer rating record
<code>complaints</code>	Ticket for post-job issues
<code>invite_codes</code>	Closed-beta access control
<code>activity_logs</code>	State changes, audit trail, admin actions

Table design notes. Keep `jobs` as the center of gravity. Important fields on `jobs` should include: `source_channel`, `pricing_mode`, `category_id`, `locality_id`, `address_text`, `customer_access_token`, `status`, `assigned_worker_id`, `quoted_amount`, `final_amount`, `payment_status`, `scheduled_at`, `started_at`, and `completed_at`. The worker profile should carry `coverage_locality_ids`, `service_category_ids`, `verification_status`, `badge_status`, `availability_status`, and cached trust fields.

Database tables not required. These should stay out of scope entirely:

Do not create now	Why not
<code>wallets</code>	No wallet product in MVP
<code>subscriptions</code>	No monetization engine yet
<code>lead_fees</code>	No lead market in MVP
<code>chat_messages</code>	No in-app chat
<code>notification_outbox</code>	Manual communication first
<code>quote_threads</code>	One quote field is enough
<code>referrals</code>	Growth loop not yet needed
<code>coupons</code>	No promo stack required
<code>ads</code>	No ad system
<code>route_tracking</code>	No live GPS
<code>invoices</code>	Payment recording is enough for beta
<code>attendance</code>	Daily-wage jobs need only start/end record
<code>analytics_events_warehouse</code>	Admin queries are enough initially

API list required. The smallest useful API surface is:

Endpoint group	Required endpoints
Public/customer	POST /public/request, GET /public/request/:token, POST /public/request/:token/rate, POST /public/request/:token/complaint, GET /public/categories, GET /public/localities
Worker	POST /worker/login, GET /worker/dashboard, GET /worker/jobs, POST /worker/jobs/:id/accept, POST /worker/jobs/:id/decline, POST /worker/jobs/:id/status
Admin	POST /admin/workers, POST /admin/workers/:id/approve, GET /admin/jobs, POST /admin/jobs/:id/dispatch, POST /admin/jobs/:id/payment, POST /admin/jobs/:id/cancel, POST /admin/invite-codes, GET /admin/metrics
Shared	POST /upload, GET /health

APIs not required. These are immediate scope traps: chat websocket APIs, route APIs, payout APIs, coupon APIs, referral APIs, subscription billing APIs, ad APIs, AI recommendation APIs, IVR APIs, and public worker search APIs.

User roles. You only need four roles:

- guest_customer
- worker_pending
- worker_verified
- admin

Anything beyond that is premature.

Booking state machine.

```
submitted
→ under_review
→ matching
→ assigned
→ in_progress
→ completed
→ payment_recorded
→ closed
```

Terminal failure branches:

```
submitted → cancelled
matching → no_worker_found
```

```
assigned → cancelled  
in_progress → disputed
```

Dispatch state machine.

```
queued  
→ shortlist_created  
→ offer_sent  
→ accepted  
→ selected  
→ assigned
```

Alternative branches:

```
offer_sent → declined  
offer_sent → expired  
accepted → not_selected  
shortlist_created → failed_no_match
```

Payment state machine.

```
unpaid  
→ amount_confirmed  
→ paid_cash | paid_upi | paid_payment_link  
→ reconciliation_pending  
→ reconciled
```

Exception path:

```
paid_* → disputed
```

Complaint state machine.

```
opened  
→ acknowledged  
→ investigating  
→ resolved | rejected
```

Optional operational end states:


```
resolved_refund
resolved_warning
resolved_worker_suspend
```

Why the audit log is mandatory. Supabase's free plan has only short log access on the platform side, and the free plan itself is quota-limited: 500 MB database size before read-only behavior, 1 GB storage, 5 GB egress, 500,000 edge-function invocations, 2 million realtime messages, and 200 peak realtime connections. That means KaamSetu should keep its own durable `activity_logs` table and should aggressively control photo uploads. I would cap uploads to **three images per request at five megabytes each**, which stays far below Supabase's free-plan file-size ceiling while preserving evidence quality. ⁶

Trust and Human Operations

Trust system v1. Trust should be operational before it becomes computational.

The worker verification flow should be: 1. Founder creates invite or preloads worker lead. 2. Worker submits onboarding form. 3. Worker uploads one ID image, one selfie, and one local reference contact. 4. Founder reviews documents and calls or checks the reference. 5. Worker is approved as `probation`. 6. After three successful complaint-free jobs, worker becomes `verified`. 7. If repeated no-shows or serious complaints appear, worker becomes `paused` or `suspended`.

This means KaamSetu v1 trust is built from **identity, locality, and behavior**, not from fancy machine-learning models. A worker badge should be one of only four visible values:

- Pending
- Probation
- Verified
- Paused

Reputation system v1. Public reputation should stay minimal. Customer-facing worker display should only show:

- worker photo
- first name
- category
- locality
- badge
- jobs completed
- average rating, only after at least three completed jobs

Internally, dispatch ordering should use a simple rules-based score:

- availability match
- locality match
- category match
- complaint flag

- response rate
- completion rate
- repeat customer count

Do not expose the full internal score. Do not build an AI ranking engine. Do not show workers a competitive leaderboard. The MVP goal is trust and reliability, not gamification.

What can remain manual. A solo-founder KaamSetu should deliberately keep these manual:

- worker document review
- reference checking
- worker approval and suspension
- assisted bookings from call and WhatsApp
- quote confirmation for large jobs
- complaint investigation
- refund or compensation decisions
- founder outreach to first workers and first customers

These are judgment-heavy tasks. Software will not improve them early; it will only hide bad decisions behind screens.

What must be automated. These must be automated from day one:

- request ID creation
- invite-code validation
- status transitions
- dispatch attempt logging
- payment ledger creation
- rating capture
- complaint ticket creation
- worker/customer audit trail
- admin metrics by job status and source channel

If the system cannot reliably tell you **what happened, when it happened, and who changed it**, then KaamSetu is not an operating system. It is just a messy inbox.

The operational corollary is important: **build software for record-keeping first, not for automation theatre.**

Tech Stack and Budget

Minimal tech stack. The cleanest solo-founder stack is:

Layer	Recommendation
Frontend	Next.js + TypeScript + responsive PWA shell
UI	Simple component library, large tap targets, icon-first actions

Layer	Recommendation
Backend	Next.js route handlers and server actions
Database	Supabase Postgres
File storage	Supabase Storage
Auth	Credentials login for worker/admin with phone + PIN; guest token links for customers
Validation	Zod or equivalent schema validation
Testing	Playwright end-to-end + basic API tests
Documentation	Markdown docs inside the repo
Analytics	Admin SQL metrics only, no warehouse

Supabase is a strong fit because it packages Postgres, Authentication, instant APIs, Edge Functions, Realtime, and Storage into one platform. For a solo founder, reducing the number of infrastructure decisions matters more than theoretical elegance. ⁷

Cheapest hosting plan. If “cheapest” means **least cash outlay**, the practical answer is:

- **Vercel Hobby** for the web app
- **Supabase Free** for database and storage
- **GitHub Free** for source control
- optionally **Cloudflare DNS/SSL** later if you want extra edge controls

Vercel’s Hobby plan is free and targeted at personal and small-scale applications, which is appropriate for a closed beta. Cloudflare Pages and Workers are also viable free-tier options, and Workers Free currently allows 100,000 requests per day, while Pages is available on all plans. The reason I would still choose Vercel first is not cost; it is **lower cognitive load** if you are building a Next.js-based product with AI assistance. If absolute vendor-cost optimization later matters more than speed of implementation, Cloudflare is a credible alternative. ⁸

Cheapest communication stack. The communication stack should be:

- WhatsApp Business app for manual assisted booking and worker contact
- one business SIM for voice calls
- no IVR
- no WhatsApp API
- no SMS OTP
- optional FCM later if you truly need push notifications

This is not only cheaper; it is also operationally cleaner. WhatsApp Business app is free and designed for small businesses, while Firebase Cloud Messaging is no-cost if you later need app notifications. In contrast, WhatsApp Business Platform is built for programmatic scale and priced by message, and Firebase phone verification for production requires billing and charges for phone auth events. ⁹

Payment architecture. Do not integrate a payment gateway in the MVP by default. Record payment first. Integrate collection later only when necessary.

The correct v1 policy is:

- default payment modes: `cash` and `direct_upi`
- system records amount, who received it, and proof if available
- if a deposit or prepayment is genuinely required, use **Razorpay Payment Links manually from the dashboard**, not a coded gateway integration

This is extremely important. Razorpay's docs explicitly support payment links without a website or app, and Razorpay's standard pricing materials state there is no setup fee and no annual maintenance charge, while successful transactions incur fees. That means KaamSetu can use online payments selectively, without building a heavy payment subsystem. ¹⁰

Maps and location policy. Do not embed Google Maps in the MVP. Use:

- locality dropdown
- address text
- landmark
- optional photo
- phone call for last-mile clarification

Google Maps Platform pricing is pay-as-you-go and cost is calculated by billable event, which is not where a solo founder should burn early budget. Map-based UX can wait until volume proves that manual address clarification is a bottleneck. ¹¹

Backups and data safety. This is one of the few places where the MVP should be slightly more disciplined than a typical solo build. Supabase documents automatic daily backups for paid plans and explicitly recommends that free-tier projects regularly export data using the Supabase CLI and maintain off-site backups. Supabase also documents automated backup workflows using GitHub Actions, and notes that database backups do **not** include files stored through the Storage API. That means KaamSetu should automate a nightly database dump and should keep uploaded media minimal and operationally important only. ¹²

Cost breakdown under the ₹10,000 budget. Based on current official pricing pages checked on June 4, 2026, your early-build infra can realistically sit at **₹0 recurring** during closed beta if you stay on free tiers for Vercel, Supabase, WhatsApp Business app, and FCM, and only use Razorpay when you intentionally send a payment link. ¹³

A practical founder cash budget is therefore:

Item	Budget assumption
Domain name	₹1,000 placeholder
Business SIM and initial recharge	₹800–₹1,200

Item	Budget assumption
Small print material or QR sheets	₹500–₹1,000
Contingency reserve	₹2,000–₹4,000
Infra and software	₹0 at closed beta if you stay on free tiers

That puts the realistic early-build envelope around **₹4,000–₹7,000**, still below the ₹10,000 ceiling. The biggest additional costs to avoid are Google Play and Apple developer enrollment, which is another reason to stay web-first until the model works. ¹⁴

Development Order and Freeze Checklist

Development order.

Step one. Build the data model and the admin shell first. The first usable milestone is not a beautiful app; it is an admin console that can create a worker, create a job, assign a job, change a job state, and record a payment.

Step two. Build worker onboarding and worker login. Without supply, there is no marketplace. Worker availability, locality coverage, service categories, and verification status must exist before customer demand is invited.

Step three. Build the customer request form and tracking page. Keep it simple, guest-first, and token-based. This is where you prevent demand-side overengineering.

Step four. Build dispatch, job lifecycle, and payment recording. This is the heart of KaamSetu. If dispatch is weak, the entire product feels fake.

Step five. Build ratings, complaints, backup automation, and beta hardening. Only after that should you start running real traffic through the system.

What the founder builds first. Before asking AI to generate code, the founder should lock these decisions:

- launch city and locality cluster
- maximum five launch categories
- worker verification checklist
- job status vocabulary
- quote policy for large jobs
- fixed-price templates for simple jobs
- complaint policy
- payment recording policy
- invite list for first workers and first customers

These are product truths. AI cannot invent them correctly from scratch.

What AI agents build first. Once those rules are locked, AI should build in this order:

- schema and migrations
- admin CRUD and filters
- worker onboarding flows
- request form and tracking page
- dispatch APIs
- job state transitions
- payment ledger UI
- automated tests for happy path and failure path
- internal docs for setup and deployment

Final MVP freeze checklist.

- ☐ One launch city, one cluster, no broad multi-city rollout
- ☐ Maximum five active service categories at beta launch
- ☐ PWA only, no native mobile apps
- ☐ No customer account system required
- ☐ Worker/admin credentials only
- ☐ No SMS OTP
- ☐ No map SDK
- ☐ No in-app chat or calling
- ☐ No wallet or escrow
- ☐ No public worker browsing
- ☐ One request engine for all channels
- ☐ One dispatch queue in admin
- ☐ Three pricing modes under one job schema
- ☐ Payment recording works for cash and UPI
- ☐ Manual WhatsApp Business app is the only messaging channel
- ☐ Ratings and complaints are live
- ☐ Activity log records every state change
- ☐ Nightly database backup is configured
- ☐ First 20 jobs will be founder-operated manually before wider beta
- ☐ Nothing ships that does not directly support onboarding, booking, matching, tracking, payment recording, or trust

Final founder recommendation. Freeze KaamSetu v1 as a **founder-operated dispatch console with two thin edges**: a guest customer request/tracking PWA and an invite-only worker dashboard. Do not build a “marketplace app.” Do not build customer auth. Do not build chat. Do not build maps. Do not build app-store binaries. Do not build payment complexity. Build a controlled job-control system that can reliably handle the first 100 jobs. If that system becomes dependable, everything else can be layered on later. If it does not, every extra feature will only make failure more expensive.

1 Progressive Web Apps

https://web.dev/learn/pwa/progressive-web-apps?utm_source=chatgpt.com

2 5 9 Download WhatsApp Business App | Get started today

https://business.whatsapp.com/products/business-app-get-started?utm_source=chatgpt.com

3 Firebase Pricing - Google

https://firebase.google.com/pricing?utm_source=chatgpt.com

4 14 Get started with Play Console

https://support.google.com/googleplay/android-developer/answer/6112435?hl=en&utm_source=chatgpt.com

6 About billing on Supabase

https://supabase.com/docs/guides/platform/billing-on-supabase?utm_source=chatgpt.com

7 Supabase | The Postgres Development Platform.

https://supabase.com/?utm_source=chatgpt.com

8 Vercel Hobby Plan

https://vercel.com/docs/plans/hobby?utm_source=chatgpt.com

10 Payment Links | FAQs

https://razorpay.com/docs/payments/payment-links/faqs/?utm_source=chatgpt.com

11 Google Maps Platform core services pricing list

https://developers.google.com/maps/billing-and-pricing/pricing?utm_source=chatgpt.com

12 Database Backups | Supabase Docs

https://supabase.com/docs/guides/platform/backups?utm_source=chatgpt.com

13 Vercel Pricing: Hobby, Pro, and Enterprise plans

https://vercel.com/pricing?utm_source=chatgpt.com